Predicting Student Placement: Model Development and Evaluation

goML AI Engineer Take-Home Assignment, Part 1

Vijay Samuel | 28 October 2026

Executive Summary. The task is to predict which registered students receive an offer through the placement cell. The metric is the F1 score for placed = 1: the harmonic mean of precision (the share of predicted placements that are correct) and recall (the share of actual placements that are found). Logistic regression is the final model in both feature tracks. Without `status_code` it reaches a nested F1 (cross-validated, with the threshold chosen on separate folds) of 0.563 (precision 0.453, recall 0.754); with `status_code` it reaches 0.924 (precision 0.929, recall 0.920). The key decision concerns `status_code`, an undocumented field that almost equals the label and is most likely written after an offer. A validity gate uses it only when the scoring file shows the same pattern as the training data. Expected F1 on the scoring file: 0.922 if `status_code` is intact, 0.554 if it is blank or unrelated to the outcome.

1 Problem and Approach

A classifier predicts `placed` for a new list of registered students. The evaluation metric is F1 for the positive class. Two constraints shape the design: the scoring file may contain empty cells in any column, and `predict.py` must load a trained model without refitting. A classification threshold is the probability above which a student is predicted as placed. Figure 1 shows the workflow.

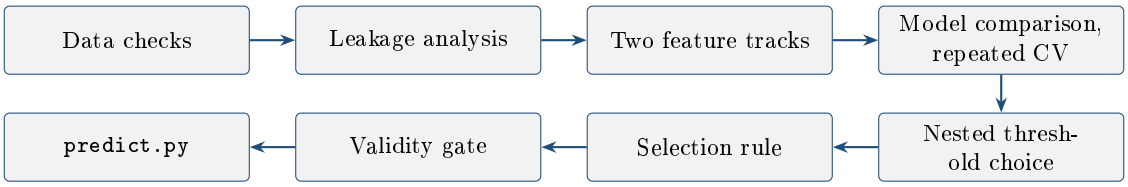


Figure 1: Workflow from data checks to the scoring script.

2 Dataset

The training file has 2,500 students and 9 candidate features. 702 students are placed (28.1%), so the positive class is the minority. Table 1 lists the columns.

3 Exploratory Data Analysis

The exploratory analysis identifies which features separate placed from unplaced students and checks data quality.

Findings.

- The strongest signals are cgpa ($r = 0.309$), coding_score ($r = 0.292$) and department (38.4% placed in CSE against 13.6% in CIVIL).
- city_tier ($r = 0.020$) and workshops ($r = 0.048$) carry little signal.
- All values lie in valid ranges. 14 students have cgpa exactly 5.0, which suggests a floor.

Column	Type	Description	Range or categories	Missing	Used
`student_id`	text	Student id	S00001 to S02500	0.0%	no
`department`	categorical	Branch	CSE, IT, ECE, EEE, MECH, CIVIL	0.0%	yes
`city_tier`	integer	City tier	1 to 3	0.0%	yes
`cgpa`	continuous	Cumulative GPA	5.0 to 10.0	4.7%	yes
`backlogs`	integer	Active backlogs	0 to 6	6.7%	yes
`internship_months`	integer	Length of internship, in months	0 to 6	0.0%	yes
`coding_score`	continuous	Internal coding test, out of 100	5.2 to 100.0	21.0%	yes
`projects`	integer	Number of projects listed	0 to 10	0.0%	yes
`workshops`	integer	Number of workshops attended	0 to 4	0.0%	yes
`status_code`	binary	A field stored on the placement portal	0 to 1	4.5%	status track only
`placed`	binary	1 if placed, 0 otherwise	0 to 1	0.0%	target

Table 1: Columns of `train.csv`. Descriptions are taken from the assignment README.

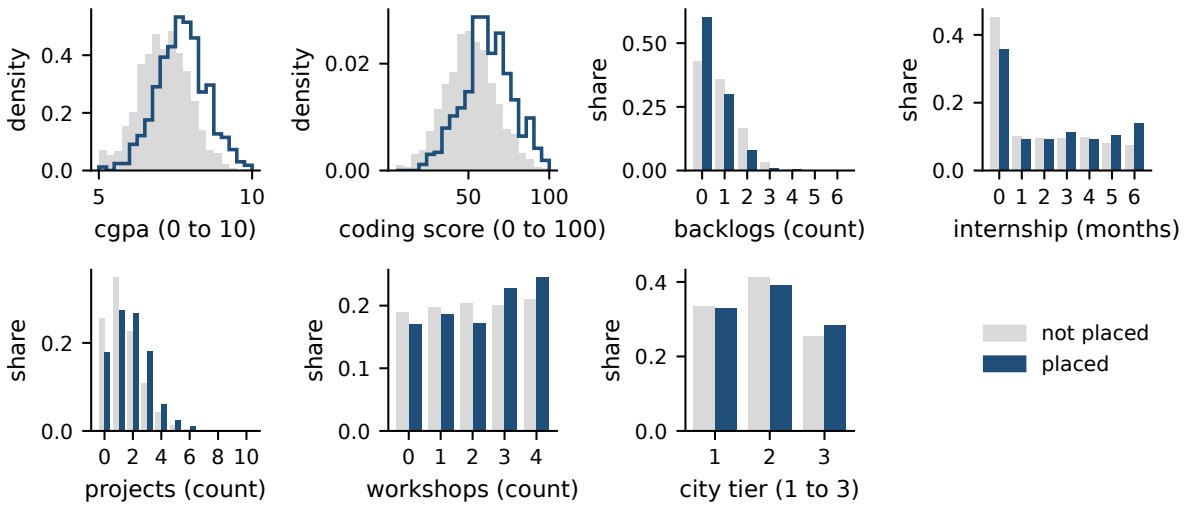


Figure 2: Distributions of numeric features by outcome. Continuous features show densities; counts show the share within each outcome.

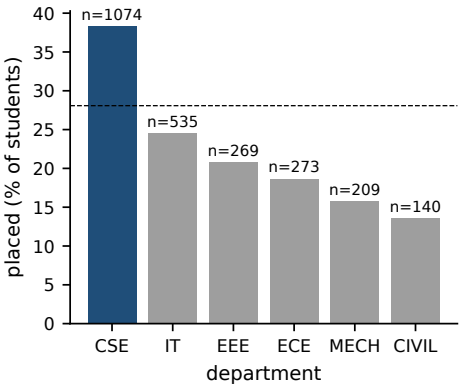


Figure 3: Placement rate by department with student counts. Dashed line: overall rate.

Column	Missing	Placed if missing	Placed if present
`cgpa`	118 (4.7%)	34.7%	27.7%
`backlogs`	168 (6.7%)	29.8%	28.0%
`coding_score`	525 (21.0%)	27.4%	28.3%
`status_code`	113 (4.5%)	21.2%	28.4%

Table 2: Missing values and placement rate when the value is missing or present.

Check	Result	Decision
Full-row duplicates	0	None to remove
Feature-row duplicates	2 rows (S02003 and S02454), same label	Kept: one pair, consistent label
Duplicate `student_id`	0	Ids are unique
Id number vs. `placed`	Pearson $r = -0.012$	Identifier with no signal, excluded

Table 3: Duplicate and identifier checks.

- Only a missing cgpa is informative (34.7% placed against 27.7%). Every numeric column receives a missing-value indicator because the scoring file may have gaps anywhere.

4 Correlation Analysis and Data Leakage

One column, `status_code`, is almost a copy of the label. Data leakage is the use of information at training time that is not available, or not valid, at prediction time [5].



Figure 4: Pearson correlations, pairwise complete. For binary columns this equals the point-biserial correlation.

`status_code`	Students	Placed	Rate
1	700	645	92.1%
0	1,687	33	2.0%
missing	113	24	21.2%

Table 4: Placement rate by `status_code`.

`status_code` in test file	Without status_code	Always with status_code	Gated system
Intact (as in training)	0.554	0.921	0.922
Shuffled	0.554	0.294	0.554
Half blank	0.554	0.664	0.719
Blank	0.554	0.200	0.554

Table 5: Stress test: mean F1 over 60 simulated test sets per scenario (Section 8).

Findings and decision.

- 92.1% of students with `status_code` = 1 are placed, against 2.0% with `status_code` = 0 ($r = 0.910$). Its correlation with cgpa is only 0.276, so it is not a merit score.
- The README defines every column except this one, and the scoring file lists newly registered students who should not yet hold offers. `status_code` is therefore treated as a probable post-outcome field.
- A model that always uses `status_code` drops to 0.294 when the field is shuffled and to 0.200 when it is blank, far below the model without it (0.554).

- Decision: two tracks with identical candidates, one without `status_code` (clean) and one with it (status), and a gate that selects the track per scoring file.

5 Methodology

Four model families are compared on identical data splits (Table 6). Neural networks are not included: on small tabular data they rarely match tree ensembles [3, 10].

Model	Reason for inclusion	Reference
Logistic regression, elastic-net	Linear model of the log-odds. Elastic-net regularization adds an L1 penalty (sets weak coefficients to zero) and an L2 penalty (shrinks all coefficients). Interpretable and well calibrated on small tabular data.	[4]
XGBoost	Gradient-boosted decision trees. Captures interactions and nonlinear effects, handles missing values natively.	[2]
LightGBM	Second boosted-tree implementation with leaf-wise growth, tuned independently.	[6]
Blend: LR + LightGBM	Average of the linear model and the stronger boosted model (LightGBM in both tracks). Tests whether the two models make complementary errors.	

Table 6: Candidate models.

Preprocessing. All steps sit inside one scikit-learn pipeline [9], so `predict.py` applies exactly the training transformations.

- Input normalization: column names matched case-insensitively, text coerced to numbers, values clipped to valid ranges.
- Median imputation with a missing-value indicator for every numeric column (tree models keep missing values as they are).
- One-hot encoding of department; unseen categories are ignored.

Validation.

- Repeated stratified 5-fold cross-validation with 5 repeats (25 folds), identical for every model and track. An out-of-fold (OOF) prediction is a prediction for a student made by a model that did not see that student.
- Hyperparameters of the boosted models are tuned on a separate split seed, with log-loss as the objective.
- Nested cross-validation for the threshold: in each fold, the threshold is chosen on the OOF predictions of the other folds of the same repeat and then applied to the held-out fold. The reported nested F1 is therefore not optimistic.
- Theoretical check: for a calibrated model, the F1-optimal threshold is about half the best F1 [7].

Selection rule. A more complex model replaces logistic regression only if it improves nested F1 by at least 0.01 and wins at least 15 of the 25 paired folds.

6 Results

Logistic regression has the highest nested F1 in both tracks, so the selection rule is never triggered (Table 7). PR-AUC (area under the precision-recall curve) and ROC-AUC (area under the receiver operating characteristic curve, 0.5 for random ranking and 1.0 for perfect ranking) are threshold-free measures of ranking quality.

(a) Clean track (without `status_code`)

Model	Nested F1	Precision	Recall	PR-AUC	ROC-AUC	Folds won
Logistic regression	**0.563 ± 0.029**	**0.453**	**0.754**	**0.570**	**0.766**	**reference**
LightGBM	0.552 ± 0.024	0.446	0.745	0.558	0.755	5 of 25
XGBoost	0.549 ± 0.029	0.451	0.748	0.562	0.758	8 of 25
Blend: LR + LightGBM	0.562 ± 0.028	0.467	0.726	0.568	0.763	10 of 25

(b) Status track (with `status_code`)

Model	Nested F1	Precision	Recall	PR-AUC	ROC-AUC	Folds won
Logistic regression	**0.924 ± 0.016**	**0.929**	**0.920**	**0.944**	**0.975**	**reference**
LightGBM	0.920 ± 0.013	0.927	0.919	0.938	0.973	5 of 25
XGBoost	0.918 ± 0.014	0.932	0.915	0.940	0.974	4 of 25
Blend: LR + LightGBM	0.922 ± 0.014	0.931	0.916	0.943	0.975	8 of 25

Table 7: Nested F1 (mean ± standard deviation over 25 folds); precision and recall from OOF predictions at each model's selected threshold; folds won against logistic regression.

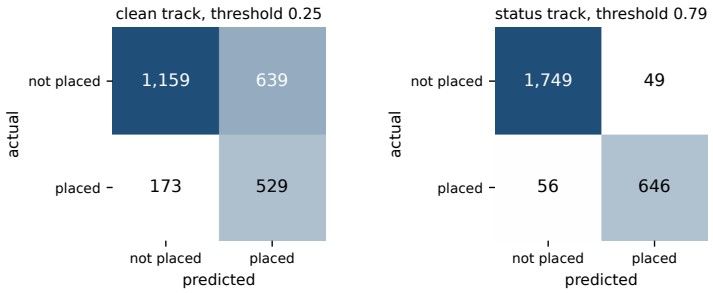


Figure 5: Confusion matrices of the shipped models (OOF predictions).

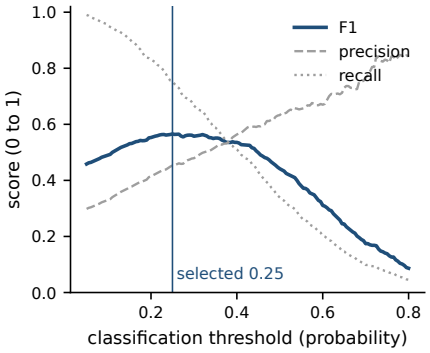


Figure 6: F1, precision and recall against the threshold, clean model.

Findings.

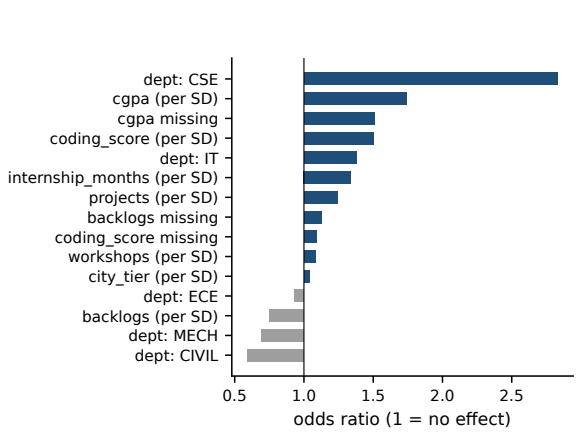
- No alternative wins more than 10 of 25 folds (clean) or 8 of 25 (status). The tuned boosted models chose trees of depth 2 to 3, which indicates little nonlinear structure.
- The selected threshold raises clean F1 from 0.442 at 0.5 to 0.566 at 0.25, close to the theoretical value 0.283.
- For the placement cell, precision 0.453 means fewer than half of flagged students are placed; recall 0.754 means most placed students are flagged.

7 Model Explainability

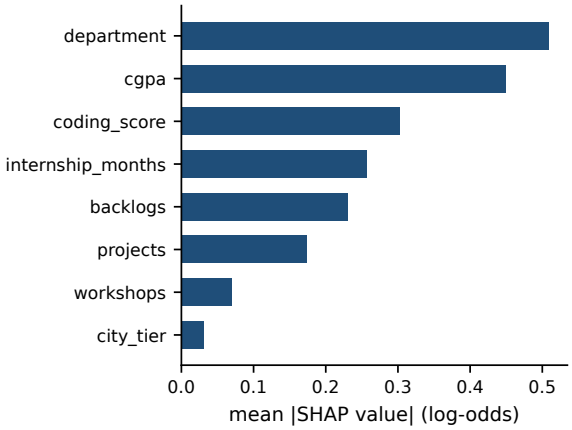
The clean logistic regression is explained with three methods that agree on the main drivers. An odds ratio is the factor by which the odds of placement change for a one-standard-deviation increase (or for the category). A SHAP value is the contribution of a feature to one prediction relative to the average prediction [8]; for a linear model it is exact and equals the coefficient times the centered feature value.

Findings.

- Main drivers in order: department, cgpa, coding_score. Permutation importance gives the same top three. CSE has the largest single odds ratio (2.83), followed by cgpa (1.74 per standard deviation).
- A missing cgpa raises the odds (1.51), consistent with Table 2.
- city_tier and workshops contribute little; they are kept because elastic-net already shrinks



(a) Odds ratios (exponentiated standardized coefficients).



(b) Mean absolute SHAP value per source feature (indicators and one-hot columns summed).

Figure 7: Global explanations of the clean model.

Feature	Mean |SHAP|	SHAP rank	Permutation drop in F1
`department`	0.509	1	0.068 ± 0.009
`cgpa`	0.450	2	0.063 ± 0.007
`coding_score`	0.303	3	0.031 ± 0.006
`backlogs`	0.231	5	0.017 ± 0.006
`internship_months`	0.257	4	0.013 ± 0.004
`projects`	0.174	6	0.007 ± 0.006
`workshops`	0.069	7	0.005 ± 0.003
`city_tier`	0.031	8	0.002 ± 0.002

Table 8: Mean absolute SHAP value and permutation importance [1]: drop in F1 at the selected threshold when a column is shuffled (10 repeats, shipped model on the training data, no refit).

Student	P(placed)	Prediction	Top contributions (log-odds)	Reading
S00802	0.565	placed	department = CSE (+0.59); projects = 5 (+0.58); internship_months = 5 (+0.41)	Raised by department, projects, internship_months.
S01956	0.060	not placed	department = MECH (−0.82); coding_score = 37.3 (−0.49); internship_months = 1 (−0.13)	Lowered by department, coding_score, internship_months.

Table 9: Two example students: the first rows in the file predicted placed and not placed. Contributions are SHAP values in log-odds.

them, and 5 coefficients are exactly zero.

- These explanations describe the model, not causal effects.

8 Deployment Logic: the Status-Code Gate

The gate decides per scoring file whether `status_code` behaves as in training (Figure 8, Table 10).

Findings.

- Gate thresholds from 0.56 to 0.70 choose the status model in at least 95% of intact runs ($n \geq 250$) and at most 5% of shuffled or blank runs. 0.63 gives the highest mean gated F1 (0.687).
- On the training file the gate AUC is 0.747 (status track); on a shuffled copy it is 0.500 (clean track).

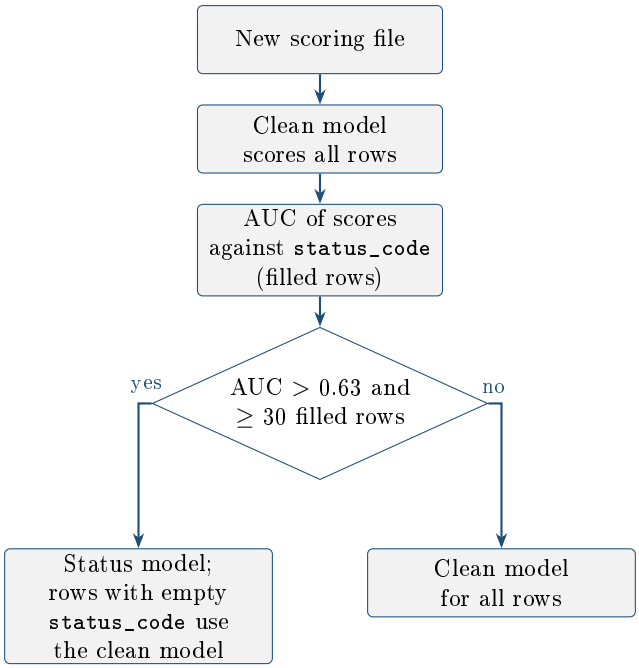


Figure 8: Gate logic in `predict.py`.

Scenario	n	Status chosen	Gated F1	Clean only F1	Always status F1
Intact (as in training)	100	100.0%	0.926	0.552	0.921
	250	100.0%	0.916	0.547	0.918
	500	100.0%	0.924	0.564	0.925
Shuffled	100	0.0%	0.552	0.552	0.308
	250	0.0%	0.547	0.547	0.278
	500	0.0%	0.564	0.564	0.297
Blank	100	0.0%	0.552	0.552	0.156
	250	0.0%	0.547	0.547	0.219
	500	0.0%	0.564	0.564	0.226
Half blank	100	95.0%	0.722	0.552	0.689
	250	100.0%	0.715	0.547	0.645
	500	100.0%	0.722	0.564	0.659

Table 10: Gate simulation: 20 stratified test sets per size; models refit on the remaining rows.

- Small files are less certain: at $n = 100$ intact files reached an AUC as low as 0.637 and shuffled files as high as 0.623.
- Blind spot: a legitimate pre-offer field that tracks merit would also pass. In production, the placement cell should confirm when `status_code` is written, which makes the gate unnecessary.

9 Robustness, Limitations, Future Work and Scalability

A robustness test blanks a random share of all feature cells in each held-out fold (Table 11). Clean F1 falls from 0.563 to 0.530 at 20% blank cells; the status track falls from 0.922 to 0.830 because blank `status_code` rows fall back to the clean model.

Track	0% blank	10% blank	20% blank	Drop at 20%
clean	0.563	0.549	0.530	0.033
status	0.922	0.876	0.830	0.093

Table 11: Robustness test: mean F1 over 25 folds with blanked cells.

Limitation	Impact	Mitigation
Small sample (2,500 rows)	Fold-to-fold F1 standard deviation 0.029; gaps below 0.01 are not distinguishable	Repeated CV, paired fold comparison, preference for the simpler model
Meaning of `status_code` unconfirmed	Expected F1 is 0.554 or 0.922 depending on the field	Validity gate; confirmation from the placement cell
F1 ceiling with available features	Clean ROC-AUC 0.766; boosted models add nothing	New information about students (future work)
Modest precision at the selected threshold	Precision 0.453: more than half of predicted placements are wrong	Raise the threshold if the cell values precision over recall
Gate variance on small files	At $n = 100$ the gate AUC of intact files fell to 0.637 and of shuffled files rose to 0.623	Minimum-row guard; manual review of small batches
Drift between batches	A different placement rate moves the F1-optimal threshold	Monitor the predicted positive rate; review the threshold per batch
Associative explanations	Coefficients and SHAP values describe the model, not causes	No interventions based on them without further study

Table 12: Limitations.

Future work	Expected benefit
Confirm when `status_code` is written	Removes the main uncertainty and the need for the gate
Additional features: aptitude scores, company tier, attendance	Raises the F1 ceiling of the clean model
Per-batch calibration and threshold review	Keeps the threshold F1-optimal when the placement rate changes
Fairness review by department and city tier	Checks error rates across groups before operational use
Monotonic constraints in boosted models	Nonlinear effects with sensible directions (cgpa up, backlogs down)
More years of data	Lower variance and a meaningful test of more flexible models

Concern	Approach
Larger data	Logistic regression and LightGBM train in time linear in the number of rows; SGD-based logistic regression or LightGBM for data beyond memory
Serving	Batch scoring with predict.py, or the same stateless scoring function behind a FastAPI endpoint
Versioning	Model bundle, thresholds and gate settings registered together in MLflow
Monitoring	Input drift, gate AUC per batch, predicted positive rate, and F1 once outcomes are known
Retraining	Scheduled retraining per placement season with the same validation protocol
Reproducibility	Pinned dependencies and fixed seeds; the full pipeline reruns to identical results

Table 13: Future work (top) and scalability (bottom).

10 Conclusion

Logistic regression with elastic-net regularization is the final model in both tracks (nested F1 0.563 without `status_code`, 0.924 with it); no boosted model, blend or stack improved on it. `status_code` is treated as a probable leak and used only when the scoring file supports it, for an expected F1 of 0.922 with an intact field and 0.554 otherwise. The next step is to confirm with the placement cell when `status_code` is recorded.

References

[1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[2] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[3] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35, Datasets and Benchmarks Track*, 2022.

[4] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2nd edition, 2009.

[5] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21, 2012.

[6] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, 2017.

[7] Zachary C. Lipton, Charles Elkan, and Balakrishnan Naryanaswamy. Optimal thresholding of classifiers to maximize F1 measure. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, pages 225–239. Springer, 2014.

[8] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30*, 2017.

[9] Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[10] Ravid Shwartz-Ziv and Amitai Armon. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90, 2022.

A Other Candidates and Feature Checks

Track	Variant	Details	Nested F1
clean	Blend: LR + XGBoost	rank-average, equal weights	0.556 ± 0.032
clean	Blend: LR + LightGBM + XGBoost	mean, LR weight 0.50	0.558 ± 0.033
clean	Stacking (LR meta-model)	LR on OOF probabilities of LR, LightGBM, XGBoost	0.555 ± 0.032
status	Blend: LR + XGBoost	mean, LR weight 0.70	0.923 ± 0.015
status	Blend: LR + LightGBM + XGBoost	rank-average, equal weights	0.921 ± 0.015
status	Stacking (LR meta-model)	LR on OOF probabilities of LR, LightGBM, XGBoost	0.921 ± 0.014
clean	LR + engineered features	cgpa × coding_score, has_internship, has_backlog	0.559
clean	LR, city_tier one-hot	same hyperparameters as the selected LR	0.559
status	LR + engineered features	cgpa × coding_score, has_internship, has_backlog	0.923
status	LR, city_tier one-hot	same hyperparameters as the selected LR	0.922

Table 14: Further variants evaluated with the same protocol (nested F1, mean ± standard deviation where available). Engineered features changed nested F1 by −0.004 (clean) and −0.002 (status), below the required gain of 0.005, so they are not used.